 ISBN：978-986-02-7580-3

Highly-Connected Community

Chen-Yi Liu and Bang Ye Wu
National Chung Cheng University, ChiaYi, Taiwan 621, R.O.C.
bangye@cs.ccu.edu.tw

Abstract

A new optimization problem "highly-connected-community" for detecting community by means of connectivity on multi-relation social networks is proposed. For nodes within a highly-connected-community, the number of uni-color disjoint paths to any other is sufficiently large. Finding maximum highly-connected-community and minimum highly-connected-community belongs to a classic problem named community detection. The difference is that, in this paper, we consider multiple relations in a social network. We propose a branch-and-bound algorithm for finding the minimum highly-connected-community, as well as an approximation and a heuristic algorithms. The experimental results show that the different algorithms have their own advantages.

Keywords: Social network analysis, algorithm, cohesion group, community, disjoint, multi-relation social network

1 Introduction

A social network can be modeled by a graph in which the nodes are actors and edges represent some kind of social relation between two actors. In social network analysis, finding cohesion groups or known as *community detection* is an important and interesting problem. In other words, we want to find an large enough node subset such that these actors in the group are closely related. For finding cohesion groups, a well-known method is to find out *clique* which is defined as a set of nodes linked to each other by an edge directly, i.e. a complete subgraph. Undeniably, clique is a natural and simple definition for cohesion group, but it is not practical at all. Finding the maximum clique in a graph is a well-known NP-hard problem [8, 19], and by the imperfect data of social networks, the too restrict definition of clique may cause the group we found is not ideal.

To overcome this difficulty, there are several relaxations have been proposed, such as k-clique, k-club, etc[6, 7, 17, 22, 24, 31, 37]. Nevertheless, either the k-clique or k-club models consider only longest shortest paths among the actors in the group but ignore the number of paths between these actors. But there are lots of applications for which we should not only consider the distance but also the number of pathways and their lengths, such as the spread of rumors and diseases. To include the above considerations, a model named by influence club, was proposed [35]. For each node within an influence club, it has a sufficiently large influence on any other in the influence club.

Besides, relationship among two actors can also be defined by the maximum flow between them[29, 30]. One may think of a flow as water owing in a network of pipes from a source to a sink. The maximum flow problem is classic and important in computer science and operating research, and many related problems have been well studies[16, 22, 24, 33]. The *connectivity*, or *node connectivity*, of two nodes is the minimum number of nodes whose removal separates the two nodes. By Menger's theory, it is equal to maximum number of disjoint paths between the two nodes and also can be thought of as a simpler from of the maximum flow between them. In social network analysis, connectivity is a basic measurement of information flow between nodes and also used to define cohesion group and centralities[15, 17, 31].

In the literatures, most research works focus on detecting communities in single-relation social networks, a simple model for social networks. But in the real network, the situation becomes more complicated, and there are usually multiple relations between two actors. When there are more than one kind of relations, a multiple-relations social network(MRSN) can be modeled by $G = (V, \{E_1, E_2, \ldots, E_c\})$, in which V is the set of nodes, $\{E_1, E_2, \ldots, E_c\}$ is a collection of c edge sets, and c is a positive integer. For $1 \leq i \leq c$, $E_i \subseteq V \times V$ represents the i-th relation, and we shall say the edges in E_i are of *color* i. Note that

there may be edges of different colors between two nodes. For a fixed c, a graph is called as a c-colors graph if there are at most c colored edge sets, and simply a "color graph" if the number of colors is not fixed or need not be specified. A path is of uni-color if all the edges of the path are of the same color. Two paths are *internally disjoint* if they have no common internal node, and a set of paths are internally disjoint if they are mutually internally disjoint. Here we shall simply use "disjoint".

In this paper we model a new optimization problem for detecting community on MRSN by means of connectivity. A *highly-connected community* is a group of actors in which any nodes has sufficiently many uni-color disjoint paths to any other in the group. In social network analysis, short paths are considered much more significant than long paths. However it is still NP-hard for finding the maximum number of uni-color disjoint paths of length bounded by 4 [36]. In this paper we only focus on uni-color paths of length at most 3.

In Section 2, we shall give some definitions about the problem of finding *maximum/minimum highly-connected community* and show how to compute the maximum number of uni-color disjoint paths of length at most 3 between two actors. In Section 3, we design some algorithms. Then we shall show the experimental results and discussions in Section 4. Finally, some conclusions are given in Section 5.

2 Preliminaries

In this section, we will discuss about the *highly-connected community* problem. First, we give some definitions and notations as follows.

- A c-color graph $G = (V, \{E_1, E_2, \ldots, E_c\})$ is a MRSN, in which V is the set of nodes, $\{E_1, E_2, \ldots, E_c\}$ is a collection of c edge sets, and c is a positive integer, and if the number of colors is not fixed or need not be specified we simply use "color graph".
- $G[S]$ is a subgraph of G induced by S, in which S is a node subset of V.
- A uni-color path is a path of which the edges are of the same color, and we shall simply use "path" in remaining paper.
- An edge $(u, v) \in E_i$ will be denoted by $(u, v; i)$, and $(v_1, v_2, \ldots, v_m; i)$ denotes a path of color i and visiting $v_1, v_2, \ldots, v_m$ in this order.
- The set of all common neighbors of nodes s and t of color i is denoted by N^i_{st}.
- The set of neighbors of nodes s of color i is denoted by N^i_s.
- $\kappa(G[S], s, t)$ represent the maximum number of disjoint paths between s and t in $G[S]$.

In Section 2.1, we introduce the minimum *highly-connected community* problem on MRSN and show the complexity of the problem. In Section 2.2, we introduce the maximum *highly-connected community* problem on MRSN. In Section 2.3, we focus on how to compute $\kappa(G[S], s, t)$, the maximum number of disjoint paths of length at most 3 between s and t in the induced subgraph $G[S]$.

2.1 Minimum highly-connected community problem

The motivation of studying the highly-connected community problem is natural. A highly-connected group ensure that two actors can communicate when some edges lose. The *highly-connected community* problem can be formally defined as follows

Definition 1 *For a given color graph G and a threshold θ, a* highly-connected community *is a subset of nodes $S \subseteq V$ such that $\kappa(G[S], s, t) \geq \theta$ for any nodes $s, t \in S$.*

Here, we define two problems for finding a highly-connected community. One is the *Min HCC* problem, and the other is the *Max HCC* problem. A *Min HCC* is defined as follows.

> Problem:A minimum highly-connected community problem on MRSN (*Min HCC*)
> Instance:A MRSN G, a positive integer θ, and a node $x \in V$.
> Goal:Find the minimum size of *HCC* containing a node x.

In some applications, finding *Min HCC* is useful and necessary. To organize a team work is an example. When we establish a department, we hope the cooperation of the members is good and minimize the cardinality of the team absolutely. A sufficiently large disjoint paths between any pair of nodes in the *HCC* represents the group can

communicate well. And to reduce the personnel expense, the size of HCC should be as few as possible in a team. In next theorem, we show that the $Min\ HCC$ problem is a NP-hard problem.

Theorem 1 *The Min HCC problem is NP-hard.*

Proof: It is a well-known NP-Hard problem to find maximum clique in a graph [8]. And with a threshold θ, the $Min\ HCC$ problem has a optimal solution of clique of size $\theta + 1$. Apparently, the $Min\ HCC$ problem is equivalent to the finding a clique of size $\theta + 1$ in a graph and is therefore NP-hard.

2.2 Maximum HCC_θ problem

A related but different problem in this paper is Max HCC problem. A Max HCC is to find the maximum cardinality of HCC. The Max HCC problem applies if the information flow or the influence spread along the same kind of relations. Computer virus spreading is a common example. One virus usually spreads only along one or several particular computer softwares, and the size of Max HCC is the infected area possibly. When there is only one color, the Max HCC problem can be solved by finding maximum θ-flow club, and there is a polynomial time algorithm has been proposed [22]. But on a c-colors graph, to our best knowledge, has not been studied yet.

2.3 Computing the connectivity

In this subsection, we show how to compute $\kappa(G[S], s, t)$, the maximum number of disjoint paths of length at most 3 between t and t in the induced subgraph $G[S]$.

Let $\kappa^l(G[S], s, t)$ denote the number of disjoint paths of length l between s and t in $G[S]$. We can rewrite the formula

$$\kappa_{\leq p}(G[S], s, t) = \sum_{l=1}^{p} \kappa^{(l)}(G[S], s, t) \tag{1}$$

The superscript "p" indicates we limit the path length to p. We shall use $p = 3$ in this paper. Note that if there are the same path of different color i, we only count once. Obviously, $\kappa^{(1)}(G[S], s, t)$=1 if there exists a edge $(s, t; i)$ for any color i.

For $l = 2$, an st-path of length 2 has the form $(s, v, t; i)$, i.e, any co-neighbor of s and t may contribute a path. And we have the following function

$$\kappa^{(2)}(G[S], s, t) = |\bigcup_i N^i_{st}| \tag{2}$$

The path of length 3 is more complicated, we use maximum matching to solve it. There is a polynomial time algorithm in [36] to find the maximum number of disjoint path of length at most 3. And we give brief introduction in the following.

Algorithm 1 finding $\kappa_{\leq 3}(G[S], s, t)$

Input: A c-colors graph G and two nodes s and t.
Output: The maximum number of disjoint unicolor st-paths of length at most 3.

1: $S \leftarrow \emptyset$; ▷ solution set
2: **for** $k \leftarrow 1$ to c **do**
3: for each node $v \in N^i_{st}$, add path $(s, v, t; i)$ into S and remove v from G;
4: **end for**
5: $F_i \leftarrow \{< u, v > |\{(s, u), (u, v), (v, t)\} \subseteq E_i\}$ for $1 \leq i \leq c$; ▷ ordered pairs
6: $F \leftarrow \bigcup_i F_i$ and construct the directed graph H induced by F;
7: find a maximum match M of H;
8: **for all** $< u, v > \in M$ **do**
9: add $(s, u, v, t; i)$ into S for some i such that $(u, v) \in F_i$;
10: **end for**
11: **return** S.

In step 5 and step 6, we get a subgraph H induced by F. Notice that we only count once on the same path $(s, u, v, t; i)$ with different color i Then, we find a maximum matching M on graph F in step 7. The cardinality of M is the maximum number of disjoint path of length 3. Therefore, we have the following formula

$$\kappa^{(3)}(G[S], s, t) = |M| \tag{3}$$

3 Algorithms

In this section, we show some algorithms. In Section 3.1, we design a a branch-and-bound algorithm and heuristic algorithm for finding Min HCC problem on a c-color graph. And a branch-and-bound algorithm for Max HCC problem is in Section 3.2.

3.1 Algorithms for Min HCC problem

In this subsection, we show a branch-and-bound algorithm for finding Min HCC problem first. And instead of only optimal solutions, our algorithm also can be an approximation algorithm for a given

ratio α. It should be noted that there may be no any feasible solution for some instances.

A configuration $(S, \bar{S}, U)$ is a partition of V into three subsets, in which S, and $\bar{S}$ resp., contain the nodes in, and not in resp., the HCC, and U is the set of the undetermined nodes. We provide a ratio α to the algorithm for a trade-off between time and qualities of solutions. We set $\alpha = 2$ in our experiments. The algorithm uses a stack to contain all configurations to be explored. So it is a depth-first-search tree-searching algorithm. Algorithm 2 is the branch-and-bound algorithm.

Algorithm 2 The branch-and-bound algorithm for finding Min HCC problem

Input: A c-colors graph G, a threshold θ, a ratio α and a nodes x.
Output: A HCC containing x with number of nodes at most $\alpha * (\theta + 1)$.

1: initialize a stack T, $Best \leftarrow \infty$;
2: push $(x, \emptyset, V)$ into T;
3: **while** T is not empty **do**
4: pop $(S, \bar{S}, U)$ from stack T;
5: check S is a feasible solution or not;
6: **if** S is feasible and $|S| < |Best|$ **then**
7: replace $Best$ by S;
8: **if** $Best \leq \alpha * (\theta + 1)$ **then**
9: **return** $Best$;
10: **end if**
11: goto step 4;
12: **end if**
13: **if** $|U| < \theta + 1 - |S|$ **then**
14: goto step 4;
15: **end if**
16: **for** each $v \in U$ and $\kappa_{\leq 3}(G[S], s, t) < \theta$ **do**
17: compute $ctb(v)$;
18: **end for**
19: compute a lower bound k which is the minimum number to be added in;
20: **if** $|U| < k$ **then**
21: goto step 4;
22: **end if**
23: **if** $|S| + k \geq (Best/\alpha)$ **then**
24: goto step 4;
25: **end if**
26: $v \leftarrow \arg\max_{v \in U} ctb(v)$;
27: push $(S, \bar{S} \cup \{v\}, U - \{v\})$ into T;
28: push $(S \cup \{v\}, \bar{S}, U - \{v\})$ into T;
29: **end while**
30: **return** $Best$.

In each iteration, we pop a configuration $(S, \bar{S}, U)$ from the stack and do the following. At Step 5 we check whether S is a feasible HCC which $\kappa_{\leq 3}(G[S], s, t) \geq \theta$ for $s, t \in S$, or not. If S is feasible and better than currently best we update it. If $|Best| \leq ratio * (\theta + 1)$, we sure return $Best$ (Step 6-10). Since $\theta + 1$ is OPT. At Step 14 we use a trivial lower bound $\theta + 1$, if $|U| < \theta + 1 - |S|$, S cannot be a feasible HCC, and we simply discard the configuration (Step 14). We compute the contribution of each $v \in U$ by following function.

$$ctb(v) = |\{s, t \in S | v \in N^i_{st}\}| + \frac{1}{2}|\{s, t \in S | \exists (s, v, u, t; i) \wedge v \notin N^i_{st}\}| \quad (4)$$

For each pair (s, t) in which $\kappa_{\leq 3}(G[S], s, t) < \theta$, if there exists a path $(s, v, t; i)$ and $(s, v, u, t; i)$ resp., then v contribute 1 and 0.5 resp., to the pair (s,t) (Step16-18). At Step 19 we use Equation (4) to compute a lower bound k, which is the minimum number of nodes to be added to satisfy the requirement. And if $|U| < k$ we simply discard this configuration. If $|S| + k \geq (Best/\alpha)$, we can also discard this configuration (Step 23-25). We then generate two configurations: one with v in S and the other with v in $\bar{S}$, v is the one we choose with maximum $ctb(v)$ for $v \in U$ so that more pairs can be satisfied.

Base on the branch-and-bound algorithm, we design a heuristic algorithm for finding a HCC as follows. S is a node subset which contain the node in the HCC, and U is the set of the undetermined nodes.

Since there is only a node x in S in the beginning, we choose a node v whose degree is maximum and put v into S. And then we do a loop as following until returning a solution or -1. First we check if S is a feasible solution or not. If S is feasible we return S. Otherwise we compute the contribution of $v \in U$ as Algorithm 2 Step 16-18. We also use a lower bound by computing k in Algorithm 2 Step 19. If there exists a solution, we put a node v, which has most contribution for nodes pair (s, t) $s, t \in S$, into S, and remove v from U. Or else we return -1. The heuristic algorithm may find a feasible HCC faster but cannot ensure that the solution is minimum. Besides, it happens that a optimal solution exists but the heuristic algorithm fails to find any feasible solution.

3.2 Algorithms for Max HCC problem

In this subsection, we give a branch-and-bound algorithm for the Max HCC problem. The algorithm is similar to the one in [35]. We introduce the algorithm briefly as follows.

Algorithm 3 The branch-and-bound algorithm for finding Max HCC problem

Input: A c-colors graph G,a threshold θ.
Output: The Max HCC covering x

1: initialize a stack T;
2: $BestClub \leftarrow \emptyset$;
3: push $(\emptyset, \emptyset, V)$ into T;
4: **while** T is not empty **do**
5: pop $(S, \bar{S}, U)$ from stack T;
6: let $H = G[S \cup U]$;
7: compute $\kappa_{\leq 3}(H, u, v)$ for each $u, v \in V(H)$;
8: **if** there exist $u, v \in S$ such that $\kappa_{\leq 3}(H, u, v) < \theta$ **then**
9: goto step 4;
10: **end if**
11: **if** $|S| + |\{u \in U | \min_{s \in S} \kappa_{\leq 3}(H, u, s) \geq \theta\}| \leq |BestClub|$ **then**
12: goto step 4;
13: **end if**
14: compute $f(v) = |\{u \in V(H) | \kappa_{\leq 3}(H, u, v) < \theta\}|$ for each $v \in V(H)$;
15: **if** $f(v) = 0$ for each $v \in V(H)$ **then**
16: replace $BestClub$ by $S \cup U$;
17: goto step 4;
18: **end if**
19: $R_1 = \{v \in U | \exists u \in S, \kappa_{\leq 3}(H, u, v) < \theta\}$;
20: **if** $R_1 \neq \emptyset$ **then**
21: push $(S, \bar{S} \cup R_1, U - R_1)$ into T;
22: goto step 4;
23: **end if**
24: $x \leftarrow \arg\max_{v \in U} f(v)$;
25: $R_2 = \{v \in U | \kappa_{\leq 3}(H, x, v) < \theta\}$;
26: push $(S, \bar{S} \cup \{x\}, U - \{x\})$ into T;
27: push $(S \cup \{x\}, \bar{S} \cup R_2, U - \{x\} - R_2)$ into T;
28: **end while**;
29: output $BestClub$.

The algorithm also use $(S, \bar{S}, U)$ to keep a configuration. And in each iteration, we pop a configuration $(S, \bar{S}, U)$ from the stack and do the following. If there exist $u, v \in S$ such that $\kappa_{\leq 3}(G[s], u, v) < \theta$, S cannot be contained in any feasible subset of H, and we simply discard the configuration (Steps 8-9). At Step 11 we compute an upper bound, if this bound is not better than the currently best, we also discard the configuration (Step 12). If the condition in Step 15 holds, $V(H)$ is a feasible club. Since it is not discarded at Step 12, it must be a better club. At Step 19, we compute R_1 as the set of nodes which cannot be in a feasible club with S. If there exists such a node, we remove them to generate a configuration which will be explored later. For otherwise, since there exists conflicting pair but no node conflicts with node in S, there must be conflicting pair of nodes in U. We then generate two configurations: one with x in S and the other with x in $\bar{S}$. If x is put into S, all nodes conflicting with x should be put into $\bar{S}$ (Step 27). The way we choose x is to maximize the conflicting nodes (R_2) so that the undetermined nodes ($|U|$) of the generated configuration is minimized.

4 Experimental results

We do experiments of Min HCC and Max HCC problem. We use Poisson random graph [1, 2]in our experiments. For a Poisson random graph, the existence of each edge has the same probability d and independent to any other. The expected density is d. We generate c-color graphs for experiments by some parameters: number of colors (c), number of nodes (n), c kind of densities for different colors. And in our experiments we set $c = 2$. We generate color graphs which the densities are all the same and the densities are all different.

4.1 Experimental results for Min HCC problem

We test several combinations of densities, and show some typical combinations in the tables.

For $n = 30$, we find the size of HCC by the branch-and-bound algorithm, the heuristic algorithm and 2-approximation algorithm with different threshold θ on each generated colors graph. We compute the average size for the found HCC of different algorithms in Table 1 and Table 3. Table 2 is the average running time for Table 1.

Table 1: Size of the same densities of $n = 30$

		OPT	$Heur.$	$2 - app.$
$D1 = 0.2$	$\theta = 3$	4.9	17.09	6.36
$D2 = 0.2$	$\theta = 4$	12.6	19.29	**
$D1 = 0.25$	$\theta = 3$	4.4	9.87	6.21
$D2 = 0.25$	$\theta = 4$	8.6	15.78	8.6
	$\theta = 5$	13.57	18.57	**
$D1 = 0.3$	$\theta = 3$	4.5	8.48	5.8
$D2 = 0.3$	$\theta = 4$	6.5	11.78	7.22
	$\theta = 5$	9.67	18.43	**

Table 2: Running time of the same densities of $n = 30$

		OPT	$Heur.$	$2 - app.$
$D1 = 0.2$	$\theta = 3$	137s	4s	98s
$D2 = 0.2$	$\theta = 4$	$> 10m$	10s	$> 10m$
$D1 = 0.25$	$\theta = 3$	135s	1s	15s
$D2 = 0.25$	$\theta = 4$	415s	5s	401s
	$\theta = 5$	$> 10m$	10s	$> 10m$
$D1 = 0.3$	$\theta = 3$	107s	1s	88s
$D2 = 0.3$	$\theta = 4$	392s	1s	204s
	$\theta = 5$	$> 10m$	3s	$> 10m$

Table 3: Size of the different densities of $n = 30$

		OPT	$Heur.$	$2 - app.$
$D1 = 0.2$	$\theta = 3$	4.22	17.09	4.3
$D2 = 0.3$	$\theta = 4$	6.33	19.29	6.86
	$\theta = 5$	8.21	27.8	9.14
$D1 = 0.3$	$\theta = 3$	4.11	6.44	7.22
$D2 = 0.4$	$\theta = 4$	5.44	11.55	7.67
	$\theta = 5$	7.3	13.1	8.67

In our experiments, we set a time limit as 10 minutes in 2-approximation algorithm since there are some hard cases in some type of input. If the running time exceeds the time limit, we break the case. It is easy to see that the size of $2 - app.$ is between the size of OPT and Heur. in every input. For example, when $D1 = 0.2, D2 = 0.2$ and $\theta = 3$ the size of OPT, Heur., and 2-app. are 4.9, 17.09 and 6.36, respectively. So it is useful to use the 2-approximation algorithm to get the trade off between time and quality of solutions. In some type of input the heuristic algorithm can provide a feasible solution fast, e.g., $D1 = 0.25, D2 = 0.25$ and $\theta = 3$. However, sometimes the heuristic algorithm may not find a feasible solution but there exists a solution in fact. The density of each color will impact the size of HCC we found, and the larger group sizes are expected as the graph density arises. With the Similar size of HCC, we find that the threshold θ is usually dependent on the total degree of a color graph. The reason may be that, if the density of a color is larger enough, then the color will contribute uni-color paths by a rate.

For $n = 100$, we compare the average size and the average running time of each case for each

density	V	Tv	Tx	X
0.04	1~2	***	***	3~
0.08	1~2	***	3~5	6~
0.1	1~2	3	4~8	9~
0.15	1~2	3~5	6~19	20~
0.2	1~2	3~11	12~27	28~
0.25	1~2	3~17	18~35	36~
0.3	1~2	3~21	23~42	43~
0.35	1~2	3~28	29~47	48~
0.4	1~3	4~31	32~52	53~

V: can find optimal solutions
X: do not exist a solution
Tv: there is a optimal solution but time out
Tx: cannot find a feasible solution in time

Figure 1: The range of solutions of the branch-and-bound algorithm with $n = 100$

found HCC by the heuristic algorithm and the 2-approximation algorithm of different threshold θ in Tables 4 and Table 5. Figure 1 is the range of solutions which can be find in time limit(10 minutes) or cannot.

Table 4: Results of the same densities of $n = 100$

		$Heur.$	$Time$	$2 - app.$	$Time$
$D1 = 0.3$	$\theta = 3$	7.2	1s	5.6	9s
$D2 = 0.3$	$\theta = 4$	11.77	1s	7.1	15s
	$\theta = 5$	15.07	1s	9.9	50s
$D1 = 0.35$	$\theta = 4$	8.97	1s	7.12	2s
$D2 = 0.35$	$\theta = 5$	12.57	2s	10.1	5s
	$\theta = 6$	14.9	1s	12.5	18s
$D1 = 0.4$	$\theta = 5$	11.53	1s	9.2	2s
$D2 = 0.4$	$\theta = 7$	12.6	2s	14.6	4s
	$\theta = 9$	19.8	3s	18.9	13s

Table 5: Results of the different densities of $n = 100$

		$Heur.$	$Time$	$2 - app.$	$Time$
$D1 = 0.1$	$\theta = 3$	9.67	1s	5.53	3s
$D2 = 0.2$	$\theta = 4$	17.78	5s	7.74	103s
$D1 = 0.1$	$\theta = 3$	6.56	1s	5.56	3s
$D2 = 0.3$	$\theta = 4$	8.78	1s	7.3	35s
	$\theta = 5$	12.67	1s	9.57	62s
$D1 = 0.2$	$\theta = 3$	6.11	1s	5.54	1s
$D2 = 0.3$	$\theta = 4$	12.67	2s	6.91	10s
	$\theta = 5$	18.67	3s	9.44	5s
$D1 = 0.2$	$\theta = 3$	6.56	1s	5.4	1s
$D2 = 0.4$	$\theta = 4$	8.56	1s	6.53	1s
	$\theta = 5$	10.89	1s	8.61	30s

In Figure 1, for the different input, V and X means that the branch-and-bound algorithm can find exact solutions and there does not exist a HCC in that case, respectively. Tv represents that there exist a Min HCC but we cannot find it in time limit. And Tx represents that we can-

 ＩＳＢＮ：978-986-02-7580-3

not find a feasible HCC in time limit since we do not know whether it exists or not. As shown in Figure 1, it is hard to find a optimal HCC in most of input when the number of nodes is large. And the α-approximation algorithm and the heuristic algorithm can really help us to get a balance between time and quality of solutions. In summary, the branch-and-bound algorithm provides a optimal solution but spends lots of time. The α-approximation algorithm gives a solution which is not bad and the running time is much more reasonable. And the heuristic algorithm give us an answer fast but cannot guarantee the size of solutions, and it may not find a solution which in fact exists sometimes.

4.2 Experimental results for Max HCC problem

For each generated color graphs, we find the maximum size of HCC of different thresholds θ of $n = 100$. We compare the average size for each found Max HCC. Figure 2 shows the experimental results.

density	0.03	0.035	0.04	0.045
0.03	θ=2 size=6.4	θ=2 size=7.1	θ=2 size=7.3	θ=2 size=9.1
0.035	x	θ=2 size=7.9	θ=2 size=8.4	θ=2 size=9.6
0.04	x	x	θ=2 size=8.6	θ=2 size=9.8
0.045	x	x	x	θ=2 size=11

Figure 2: The Max HCC of n=100

For the Max HCC, it can be expected that the size of Max HCC raises while the graph density increases. In some type of input, the Max HCC has size very small or very large. When the densities of colors is 0.1/0.1 and threshold $\theta = 4$, we can find the Max HCC of average size 96.5, but the size of Max HCC become 0 when the threshold $\theta = 5$. The reason is that, for relative high density, the number of disjoint paths increases rapidly as the size increases. In the meantime, for a higher threshold θ, there is no HCC of small size can exist because there are not sufficiently many disjoint paths. As a result, for such situations, the Max HCC are either relative large or contains only one node.

5 Concluding remarks

In this paper, we model a new optimization problem (HCC) for detecting community on multi-relation social networks. For nodes within a HCC, their disjoint paths are sufficiently enough. We design a branch-and-bound algorithm for finding the HCC of minimum size, and a α-approximation, a heuristic algorithm for finding HCC. The experimental results show that the different algorithm has its own advantages.

Acknowledgment

This work was supported in part by NSC 97-2221-E-194-064-MY3 and NSC 98-2221-E-194-027-MY3 from the National Science Council, Taiwan.

References

[1] R. Solomonoff and A. Rapoport, Connectivity of random nets, *Bulletin of Mathematical Biophysics*, 13:107–117, 1951.

[2] P. Erdös and A. Rényi, On random graphs, *Publicationes Mathematicae*, 6:290–297, 1959.

[3] D.J. de S. Price, Networks of scientific papers, *Science*, 149:510–515, 1965.

[4] C.H. Hubbell, An input-output approach to clique detection, *Sociometry*, 28:277–299, 1965.

[5] Dinic, E.A.: Algorithm for solution of a problem of maximum flow in networks with power estimation. Sov. Math. Dokl. II, 1277–1280 (1970)

[6] R.D. Alba, A graph-theoretic definition of a sociometric clique. *Journal of Mathematical Sociology*, 3:113–126, 1973.

[7] R.J. Mokken, Cliques, clubs, and clans, *Quality and Quantity*, 13:161–173, 1979.

[8] Garey, M.R., Johnson, D.S: Computers and Intractability: A Guide to The Theory of NP-Completeness. Freeman, NewYork (1979)

[9] L.C. Freeman "Centrality in networks: I. Conceptual clarification", *Social Networks* 1979 pp.215–239.

[10] Micali, S., Vazirani, V.V.: An $O(\sqrt{|V|}|E|)$ algorithm for finding maximum matching in general graphs. FOCS, 17–27 (1980)

[11] Seymour, P.D.: Disjoint paths in graphs. Discret. Math. 29, 293–309 (1980)

[12] Shiloach Y.: A polynomial solution to the undirected two paths problem. J. ACM 27, 445–456 (1980)

[13] Tovey, C.A.: A simplified NP-complete satisfiability problem. Discret. Appl. Math. 8(1), 85–89 (1984)

[14] McHugh, J.A.: Algorithmic Graph Theory. Prentice Hall (1990)

[15] L.C. Freeman, S.P. Borgatti and D.R. White, Centrality in valued graphs: A measure of betweenness based on network flow,*Social Networks*, 13(2):141–154, 1991.

[16] R.K. Ahuja, T.L. Magnanti and J.B. Orlin "Network Flow Theory, Algorithm, And Applications" (1991)

[17] Wasserman S., Faust, K.: Social Network Analysis, Cambridge University Press, Cambridge (1994)

[18] Robertson, N., Seymour, P.D.: Graph minors. XIII. The disjoint paths problem. J. Comb. Theory, Series B 63, 65–110 (1995)

[19] D.S. Johnson and M.A. Trick, editors, *Cliques, coloring, and Satisability: Second DIMACS Implementation Challenge*, DIMACS Series in Discrete Mathematics and Theoretical Computer Science. American Mathematical Society, 1996.

[20] M. Stoer and F. Wangner "A Simple Min-Cut Algorithm", *ACM*, 1997 pp.585–591.

[21] V.Guruswami, S.Khanna, R. Rajaraman, B. Shepherd and M. Yannakakis "Near-Optimal Hardness Results and Approximation Algorithm for Edge-Disjoint Paths and Related Problems". *STOC'99* 1999 pp.19–28

[22] J.M. Bourjolly, G. Laporte and G. Pesant, Heuristics for finding k-clubs in an undirected graph, *Computers Operations Research*, 27:559–569, 2000.

[23] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. MIT Press and McGraw-Hill (2001)

[24] J.M. Bourjolly, G. Laporte and G. Pesant, An exact algorithm for the maximum k-club problem in an undirected graph, *European Journal of Operational Research*, 138:21–28, 2002.

[25] M.E.J. Newman, The structure and function of complex networks, *SIAM Review*, 45:167–256, 2003.

[26] G. Baier "Flows with Path Restrictions", *PhD thesis, TU Berlin* 2003

[27] S.P. Borgatti "Centrality and network flow" *Social Network* 2004

[28] Yuan, S., Varma, S., Jue, J.P.: Minimum-color path problems for reliability in mesh networks. IEEE INFOCOM 2005 volume 4, 2658–2669 (2005)

[29] U. Brandes and D. Fleischer, Centrality measures based on current flow, in *Proceedings of the 22nd International Symposium on Theoretical Aspects of Computer Science* (STACS 05), *Lecture Notes in Computer Science*, 3404: 533–544, 2005.

[30] S.P. Borgatti, Centrality and network flow, *Social Networks*, 27:55–71, 2005.

[31] Hanneman, R.A., Riddle, M.: Introduction to Social Network Methods, http://www.faculty.ucr.edu/hanneman/nettext/ (2005)

[32] Hassin, R., Monnot, J., Segev D.: Approximation algorithms and hardness results for labeled connectivity problems. J. Comb. Optim. 14(4), 437–453 (2007)

[33] C.-P. Yang, H.-C. Chen, S.-D. Hsiea and B.Y. Wu, Heuristic Algorithms for finding 2-clubs in an undirected graph, *NCS*, Taiwan, 2009.

[34] Gutin, G., Kim, E.J.: Properly coloured cycles and paths: results and open problems. Golumbic Festschrift, LNCS 5420, 200–208 (2009)

[35] C.-P. Yang, C.-Y. Liu, and B.Y. Wu, Influence Clubs in Social Networks, *ICCCI* 2010.

ＩＳＢＮ：978-986-02-7580-3

[36] Wu, B.Y.:The maximum disjoint paths problem on multi-relations social networks, manuscript, 2011.

[37] UCINet, Release 6.0, Mathematical Social Science Group, Social School of Science, University of California at Irvine.